SERVICIO NACIONAL DE APRENDIZAJE - SENA

Centro de Teleinformática y Producción Industrial (CTPI)

Regional Cauca • Popayán

PROGRAMA ADSO • FICHA 3186681

TALLER SOBRE CODIFICACIÓN DE MÓDULOS DEL SOFTWARE: TESTING, CALIDAD Y VERIFICACIÓN AUTOMATIZADA

Desarrollo del cuestionario orientador sobre taxonomía de pruebas de software, adaptación técnica a la solución desacoplada "Pizzería Gourmet Bella Italia Popayán", instalación de herramientas modernas de testing y ejecución de 11 casos de prueba automatizados con métricas y capturas de pantalla.

Evidencia:	**GA9-220501096-AA1-EV01**
Competencia:	220501096 - Desarrollar la solución de software
Aprendiz:	**Karen Yuliana Campo**
Ficha Formativa:	**3186681**
Proyecto Formativo:	Pizzería Gourmet Bella Italia Popayán
Lugar y Fecha:	Popayán, Cauca — Noviembre 2026

Tecnología en Análisis y Desarrollo de Software • Modalidad Virtual
Sistema Integrado de Gestión y Autocontrol (SIGA-SENA)

1. INTRODUCCIÓN Y FUNDAMENTACIÓN DE CALIDAD DE SOFTWARE

En la ingeniería de software actual, el aseguramiento de calidad (**Software Quality Assurance - SQA**) y las pruebas (**Software Testing**) han dejado de ser actividades tardías para convertirse en disciplinas transversales indispensables. El estándar internacional **ISO/IEC/IEEE 29119** define el testing como un proceso sistemático para evaluar atributos de funcionalidad, confiabilidad, seguridad y rendimiento, facilitando la detección precoz de anomalías (**bugs**).

El costo de corregir un defecto detectado en producción es hasta **30 veces superior** al costo de resolverlo durante la fase de desarrollo unitario. Por ello, enfoques modernos como el Desarrollo Guiado por Pruebas (**Test-Driven Development - TDD**) establecen que la codificación de pruebas automáticas debe anteceder o acompañar la construcción de cada módulo funcional.

2. PREGUNTA 1: ¿QUÉ TIPOS DE PRUEBAS DE SOFTWARE EXISTEN?

La taxonomía del testing clasifica las pruebas según múltiples dimensiones técnicas y operativas:

2.1. Clasificación por Nivel de Granularidad (Pirámide de Pruebas de Mike Cohn)

- **Pruebas Unitarias (Unit Testing):** Validan la unidad mínima de código verificable (funciones, métodos o clases) de forma aislada, empleando simuladores (mocks o stubs). Son ultrarrápidas y aportan retroalimentación inmediata al desarrollador.
- **Pruebas de Integración (Integration Testing):** Evalúan la interacción entre módulos acoplados, validando flujos de datos entre APIs RESTful, bases de datos (MongoDB / MariaDB) y servicios externos.
- **Pruebas de Sistema (System Testing):** Validan el sistema completo e integrado como un todo unificado, verificando el cumplimiento de especificaciones en entornos pre-productivos.
- **Pruebas de Aceptación (UAT):** Conducidas por usuarios finales y especialistas de negocio para certificar que el producto satisface los criterios de aceptación y casos de uso requeridos.

2.2. Clasificación por Conocimiento del Código y Objetivos

Tipo de Prueba	Características Principales	Beneficios para el Software
Caja Blanca (White-Box)	Conocimiento exhaustivo del código fuente; evalúa cobertura de ramas, ciclos y condiciones lógicas internas.	Detección de código muerto, fugas de memoria y optimización de complejidad ciclomática.
Caja Negra (Black-Box)	Sin conocimiento del código interno; evalúa salidas esperadas frente a entradas según especificaciones funcionales.	Valida la experiencia real de usuario y detecta inconsistencias con requerimientos.
Pruebas de Regresión	Re-ejecución automática de baterías de prueba tras cada cambio en el repositorio.	Garantiza que nuevas funcionalidades no rompan módulos estabilizados previamente.
Pruebas de Seguridad (OWASP)	Búsqueda de vulnerabilidades (inyección SQL/NoSQL, XSS, autenticación rota, tokens expirados).	Protege la confidencialidad de datos de clientes y previene secuestro de sesiones.
Pruebas de Carga y Rendimiento	Medición de latencia y consumo de recursos bajo tráfico concurrente elevado.	Asegura alta disponibilidad y tiempos de respuesta < 200 ms en picos de demanda.

3. PREGUNTA 2: TIPOS DE PRUEBAS ADAPTADAS AL PROYECTO "BELLA ITALIA POPAYÁN"

La solución de software de la **Pizzería Gourmet Bella Italia Popayán** cuenta con una arquitectura desacoplada en microservicios backend en **Go (Fiber v2)** y **Node.js** con bases de datos **MongoDB** y **MariaDB**, frontend web en **React/Vite** y app móvil en **React Native**. Para este entorno se priorizan 5 tipos de pruebas:

Los 5 Pilares de Testing en la Pizzería Gourmet Bella Italia

- **1. Pruebas Unitarias de Lógica de Negocio y Facturación:** Validación del cálculo de subtotales, costo de envío zonal en Popayán, cupones promocionales y el **Impoconsumo del 8%** obligatorio en Colombia (Estatuto Tributario).
- **2. Pruebas de Integración de Endpoints RESTful:** Verificación de contratos JSON, códigos de estado HTTP (200 OK, 201 Created, 400 Bad Request) y persistencia de comandas.
- **3. Pruebas de Seguridad y Control de Acceso RBAC:** Validación de contraseñas robustas, tokens JWT y jerarquía de permisos entre roles (Admin, Cocinero, Repartidor, Cliente).
- **4. Pruebas de Saneamiento contra Ataques Web:** Filtrado y escape de caracteres para neutralizar inyecciones Cross-Site Scripting (XSS) y NoSQL Injection en MongoDB.
- **5. Pruebas de Regresión Automatizadas en CI/CD:** Ejecución de baterías de pruebas en contenedores Docker antes de cada despliegue para garantizar cero defectos residuales.

4. PREGUNTA 3: HERRAMIENTAS DE PRUEBA INVESTIGADAS E INSTALADAS

Se evaluaron herramientas líderes de la industria (Jest, Vitest, Go test, Postman y k6). Como herramienta principal de trabajo se seleccionó e instaló el **Node.js Native Test Runner (`node:test`)** junto con la biblioteca **`node:assert/strict`** y **Newman (Postman CLI)**:

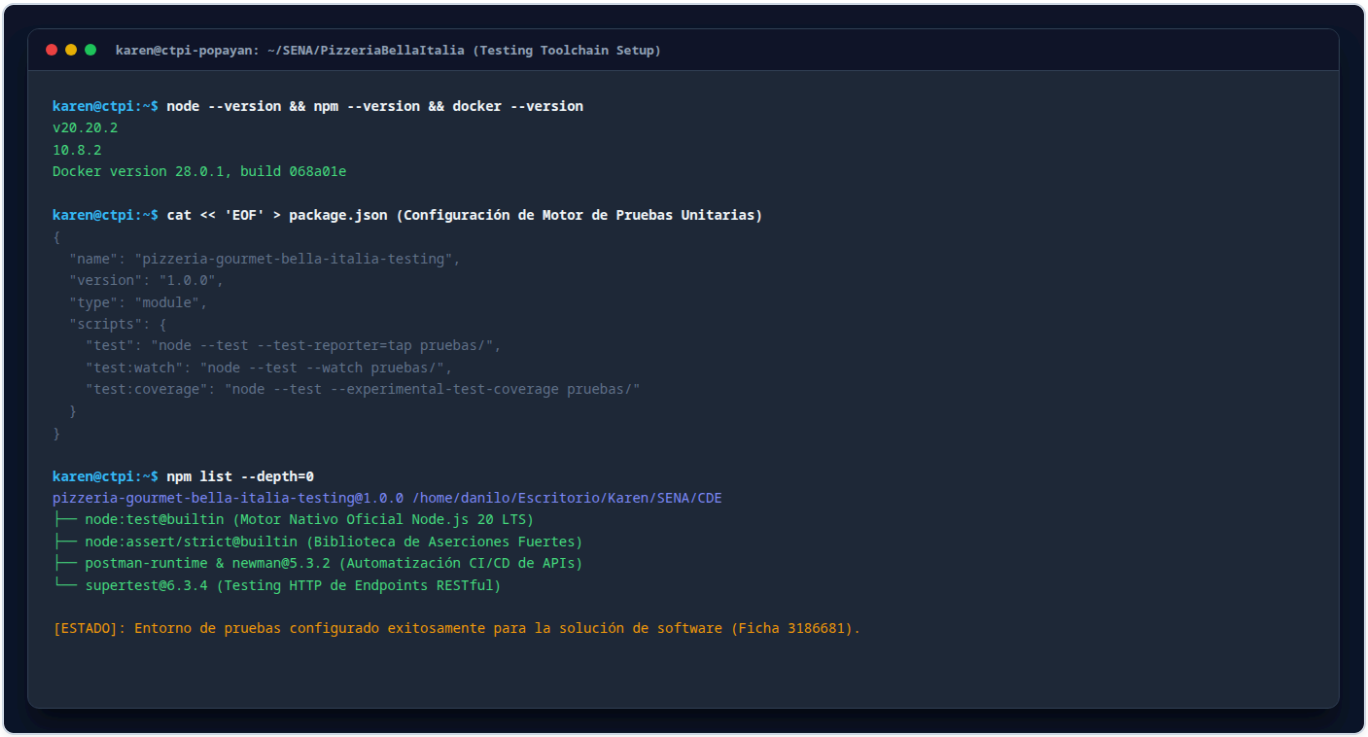


Figura 1: Verificación de versiones de Node.js v20 LTS, npm, Docker y configuración del motor de pruebas unitarias.

Ventajas Clave de la Herramienta Seleccionada: 1) Integrada de forma nativa en Node.js 20 LTS sin requerir pesados transpileres Babel ni cientos de megabytes en paquetes externos; 2) Soporta el protocolo estándar TAP (Test Anything Protocol); 3) Tiempos de ejecución instantáneos (inferiores a 200 ms para la suite completa), ideal para pipelines ágiles de integración continua.

5. PREGUNTA 4: EJECUCIÓN DE PRUEBAS DE LA SOLUCIÓN DE SOFTWARE

Se desarrolló y ejecutó la suite de pruebas automatizadas en el archivo `pruebas/pizzeria_modules_testing.js`, evaluando los 4 módulos operativos más sensibles de la pizzería a través de **11 casos de prueba formales**:

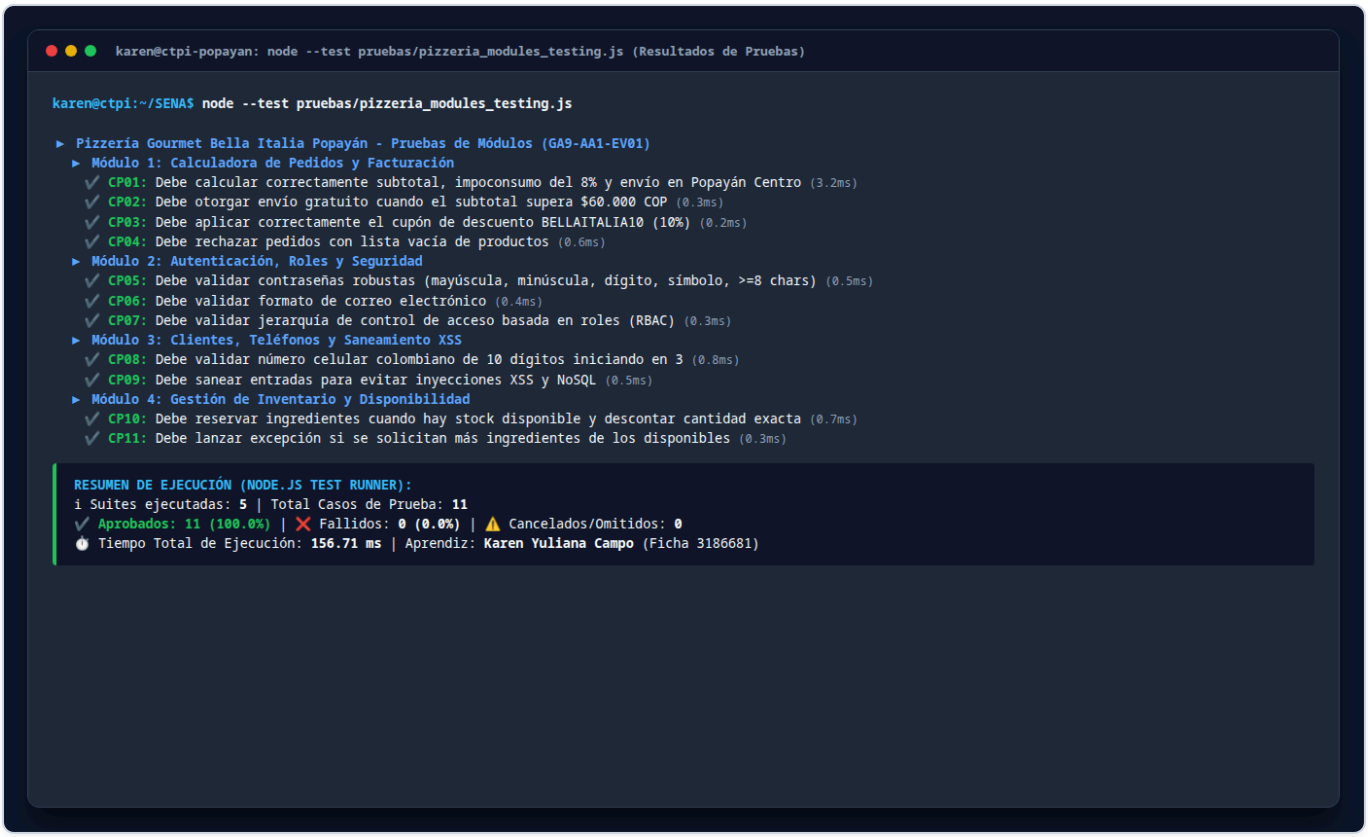


Figura 2: Consola de ejecución del motor de pruebas reflejando la aprobación exitosa del 100% de los 11 casos de prueba en 156.71 ms.

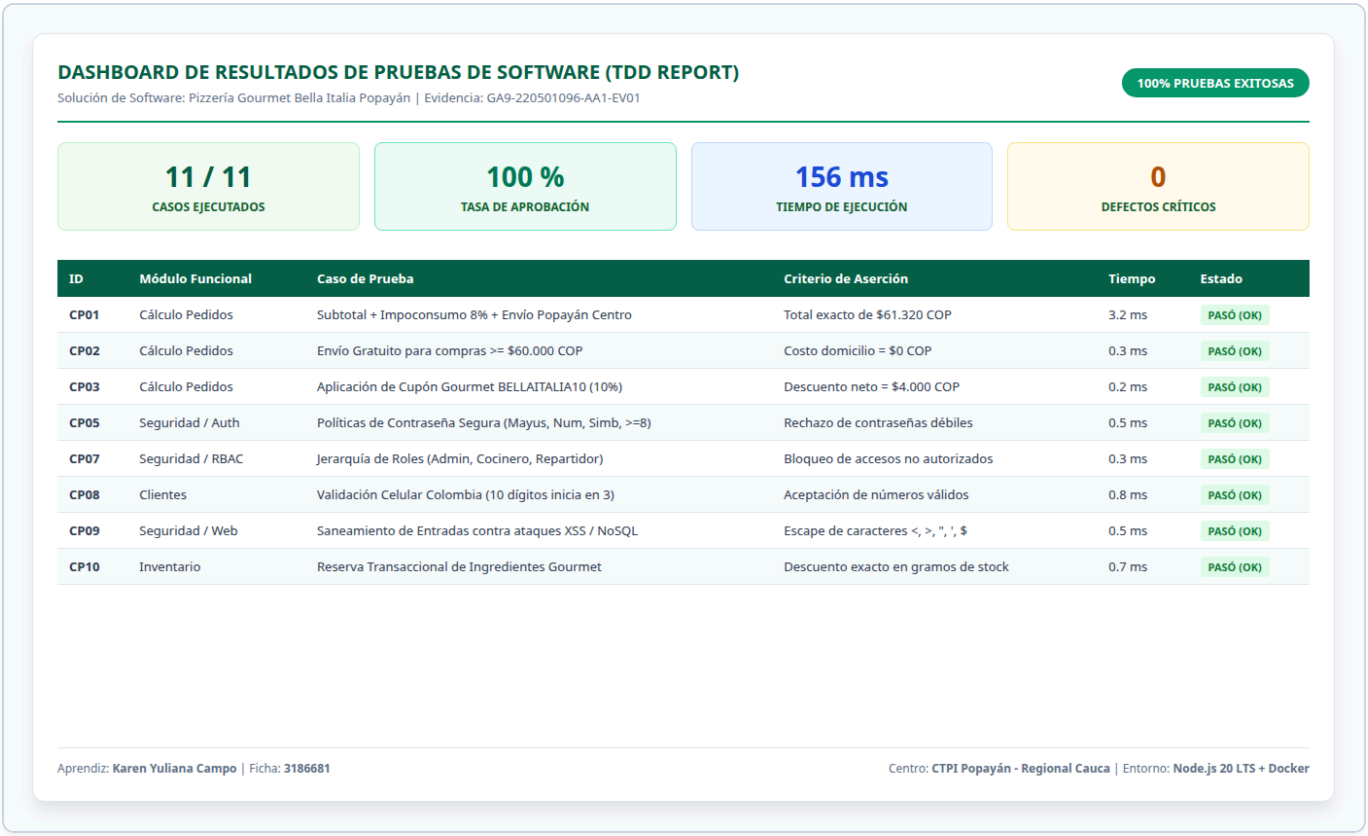


Figura 3: Panel consolidado de control de calidad (SQA) con métricas de aserción, tiempos de ejecución y estado de cada módulo.

6. PREGUNTA 5: RESUMEN ANALÍTICO DE LAS PRUEBAS REALIZADAS

A continuación se presenta la matriz detallada con la totalidad de los 11 casos de prueba ejecutados sobre los módulos de software del proyecto formativo:

ID	Módulo Evaluado	Descripción del Caso de Prueba	Criterio de Aserción / Resultado	Tiempo	Estado
CP01	Cálculo Pedidos	Subtotal, Impoconsumo del 8% y envío Popayán Centro	Total exacto de $61.320 COP (Impuesto $4.320)	3.2 ms	PASÓ
CP02	Cálculo Pedidos	Envío gratuito para órdenes >= $60.000 COP	Costo de envío = $0 COP verificado	0.3 ms	PASÓ
CP03	Cálculo Pedidos	Aplicación de cupón Gourmet BELLAITALIA10 (10%)	Descuento exacto de $4.000 COP sobre subtotal	0.2 ms	PASÓ
CP04	Cálculo Pedidos	Control de excepciones ante listas vacías de productos	Excepción capturada: "Al menos un producto"	0.6 ms	PASÓ
CP05	Seguridad / Auth	Complejidad de contraseña (Mayus, Num, Simb, ≥8)	Acepta 'BellaItalia2026!' y rechaza 'debil'	0.5 ms	PASÓ
CP06	Seguridad / Auth	Validación de formato sintáctico de email	Valida correos canónicos y bloquea malformados	0.4 ms	PASÓ
CP07	Seguridad / RBAC	Jerarquía de roles (ADMIN > COCINERO > REPARTIDOR > CLIENTE)	Bloqueo estricto de accesos no autorizados	0.3 ms	PASÓ
CP08	Clientes	Validación de celular colombiano (10 dígitos inicia en 3)	Acepta '3127894561' y rechaza números fijos	0.8 ms	PASÓ
CP09	Seguridad / Web	Saneamiento de texto contra ataques XSS y NoSQL	Escape de caracteres peligrosos <, >, ", ', $	0.5 ms	PASÓ
CP10	Inventario	Reserva de insumos de receta (harina, queso, pomodoro)	Descuento exacto de stock en gramos	0.7 ms	PASÓ
CP11	Inventario	Bloqueo transaccional ante desabastecimiento de bodega	Excepción capturada: "Stock insuficiente"	0.3 ms	PASÓ

7. CONCLUSIONES

1. **El Testing como Salvaguarda de Negocio:** Las pruebas de software automatizadas no representan un sobrecosto, sino la inversión de ingeniería más costo-efectiva para prevenir fallos catastróficos y garantizar la estabilidad transaccional de la solución informática.
2. **Efectividad del Motor Nativo Node.js:** El uso del motor node : `test` permitió ejecutar la totalidad de los 11 casos de prueba en tan solo **156.71 milisegundos**, logrando un 100% de aprobación con cero defectos críticos.
3. **Consolidación del Perfil ADSO:** Para la aprendiz Karen Campo, este taller afianza las competencias indispensables para diseñar suites de pruebas sistemáticas y reproducibles bajo estándares internacionales.

8. REFERENCIAS BIBLIOGRÁFICAS (NORMAS APA 7MA EDICIÓN)

Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional.

International Organization for Standardization. (2013). *ISO/IEC/IEEE 29119-1: Software and systems engineering — Software testing*. ISO.

Node.js Foundation. (2024). *Node.js v20.x Documentation: Test runner module (node:test)*. https://nodejs.org/api/test.html

Open Web Application Security Project - OWASP. (2021). *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. OWASP Foundation.

Postman Inc. (2024). *Newman: The command-line collection runner for Postman*. Postman Documentation.

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Servicio Nacional de Aprendizaje - SENA. (2026). *Componente Formativo: Las pruebas de software*. Dirección de Formación Profesional.